

task_bridge

Your task board and your source-of-truth, impeccably in sync — both ways.

Summary

task_bridge is a generic, decoupled, bidirectional task/board sync engine in Go. It keeps a project's workable-items SQLite source-of-truth in sync with its tracker docs and a remote board (first target: ClickUp; Jira/Linear planned) using deterministic last-edit-wins, dry-run-first, never-corrupt semantics.

Short description

A project-agnostic Go submodule that bidirectionally syncs a SQLite workable-items SSoT ↔ tracker docs ↔ a remote board (ClickUp first). Deterministic last-edit-wins, dry-run-first, HMAC-verified webhooks; every credential and ID is injected by the consumer at runtime.

Long description

Every team eventually runs two ledgers of the same work: the real one — code, docs, an internal database — and the one managers watch, a board like ClickUp. The two drift apart the instant either side is touched, and reconciling them by hand is exactly the kind of tedious, error-prone chore nobody does reliably. task_bridge is built to erase that gap by treating all three representations as one system to be held in lockstep: a project's **workable-items SQLite single-source-of-truth**, its **tracker documentation**, and a **remote board** — the first supported board being ClickUp, with Jira and Linear planned as future members. Synchronization is deterministic (last-edit-wins), dry-run-first, and engineered around a single non-negotiable promise: it will never corrupt or lose data, and never silently leave one side stale. In a domain where a careless sync can overwrite a week of work, that safety posture is the entire point. Architecturally it is a strict submodule consumed by other projects and is fully project-agnostic per the

constitution's decoupling contract (§11.4.28): it ships zero project-specific values, and every credential, board/folder ID, item-key field, and DB path is injected by the consumer at runtime through `pkg/config.Config`. The module is cleanly layered: a CLI (`reconcile/push/pull/resolve/status/conflicts/init`) and a long-running daemon (webhook receiver + cron reconcile); a thin client wrapper over the MIT-licensed `raksul/go-clickup`; a resolver that turns board/folder URLs into IDs via live API probes (no URL-grammar guessing); a mapper between local workable items and remote task fields; a last-edit-wins sync engine with explicit conflict outcomes; and a webhook receiver that verifies X-Signature HMAC-SHA256. It is honest about maturity: this is the P1 scaffold — layout, interfaces, entrypoints, and the decoupling boundary are in place, but sync logic and live ClickUp calls are not yet implemented (every stub returns an explicit not-implemented error, per the no-fakes rule).

Why we built it

Teams keep the “real” state of work in code/docs while managers live on a board like ClickUp — and the two diverge constantly. `task_bridge` makes them one system, syncing deterministically and safely so neither side becomes stale or wrong.

Why it's a game-changer

Two-way board sync is normally a one-off, hard-wired integration that every team rebuilds badly. `task_bridge` reframes it as a reusable, credential-injected library with strict data-safety guarantees baked in — dry-run-first, deterministic last-edit-wins, HMAC-verified events — so any project can adopt trustworthy board integration by injecting config rather than by writing yet another fragile connector coupled to its internals.

What's innovative

- Three-way bidirectional sync: SQLite SSoT ↔ tracker docs ↔ remote board.
- Total decoupling (§11.4.28): zero project values; all injected at runtime.
- Live-API URL→ID resolution instead of fragile URL-grammar parsing.
- HMAC-SHA256-verified webhook ingestion for live events.

Challenges & solutions

- **Data safety across three sources:** solved with deterministic last-edit-wins, dry-run-first, and explicit conflict outcomes.
- **Reusability without coupling:** solved via the pkg/config injection boundary (no shipped project specifics).
- **Reliable board identification:** solved by resolving URLs to IDs through live API probes.
- **Honest scaffolding:** solved by making unimplemented stubs return explicit not-implemented errors (no fakes).

Tech stack (why + how)

- **Go** — engine, CLI (cmd/task_bridge), and daemon (cmd/task_bridged).
- **SQLite** — the workable-items single-source-of-truth.
- **raksul/go-clickup (MIT)** — ClickUp transport wrapper.
- **HMAC-SHA256** — webhook signature verification.
- **cron + webhooks** — daemon reconcile + live-event ingestion.
- **pkg/config** — runtime credential/ID injection boundary.

Status honesty: this is a **P1 scaffold** — sync logic is not yet implemented. Do not present as shipped.